

A
PROJECT REPORT ON
IMAGE TO SKETCH APPLICATION USING AI
SUBMITTED IN
STATE BOARD OF TECHNICAL EDUCATION AND TRAINING
PARTIAL FULFILLMENT FOR THE AWARD OF THE DEGREE OF DIPLOMA
IN
COMPUTER ENGINEERING

BY
Y. BHANU SAI (23608-CM-064)
G. NANDA KISHOR (23608-CM-013)
K. AKASH (23608-CM-071)
G. BHASKAR (23608-CM-010)

UNDER THE ESTEEMED GUIDENCE OF

M. PRASADA RAO M.Tech



KALLAM HARANADHA REDDY INSTITUTE OF TECHNOLOGY
(AUTONOMUS)

(APPROVED BY AICTE NEW DELHI, AFFILIATED TO JNTUK-KAKINADA)
ACCREDITED BY NAAC WITH 'A' GRADE AND ACCREDITED BY NBA.
NH-16, CHOWDAVARAM, GUNTUR-522019.

KALLAM HARANADHA REDDY INSTITUTE OF TECHNOLOGY

(AUTONOMUS)

(APPROVED BY AICTE NEW DELHI, AFFILIATED TO JNTUK-KAKINADA)

ACCREDITED BY NAAC WITH 'A' GRADE AND ACCREDITED BY NBA

NH-16, CHOWDAVARAM, GUNTUR-522019.

COMPUTER ENGINEERING

CERTIFICATE



This is to certify that the project report entitled “**IMAGE TO SKETCH APPLICATION USING AI**” is submitted by **Y. BHANU SAI(23608-CM-064), G. NANDA KISHOR(23608-CM-013), K.AKASH (23608-CM-071), G. BHASKAR(23608-CM-010)**, to the state board of technical education, Vijayawada in partial fulfilment for the award of degree of Diploma in Computer Engineering is benefited record of project work carried out by them under my supervision during the academic year 2023-2024.

PROJECT GUIDE

M. PRASADA RAO M.Tech

Assistant Professor

HEAD OF THE SECTION

K. NAGESWAR RAO M.Tech

Associate Professor

EXTERNAL EXAMINER

DECLARATION

We hereby to declare that the entire project work embodied in this dissertation entitled **“IMAGE TO SKETCH APPLICATION USING AI”** been independently carried out by us. As per our knowledge, no part of this work has submitted for any degree in any institution, University & organization previously. We here by boldly state that to the best of my knowledge my work is free from plagiarism.

BATCH NO : 06

- | | |
|-------------------|----------------|
| 1. Y.BHANU SAI | (23608-CM-064) |
| 2. G.NANDA KISHOR | (23608-CM-013) |
| 3. K.AKASH | (23608-CM-071) |
| 4. G.BHASKAR | (23608-CM-010) |

ACKNOWLEDGEMENT

We proudly express our gratitude and respect towards our honourable chairman is **KALLAM MOHAN REDDY**, chairman of Kallam group for his precious support in the college.

We are thankful to **Dr. M.UMASHANKAR REDDY**, Director, for his encouragement and support for the completion of the project. We are inspired a lot through his valuable message.

We express our great pleasure to **Dr. B.S.B. REDDY**, Principal, KHIT, GUNTUR, for his support during and till the completion of the project.

We are really thankful to Mr. K.NAGESWAR RAO, Head of the Section of CME, for providing the laboratory facilities to the fullest extent as and when required and also for giving us this opportunity to carried out this project work in the college.

We are really thankful to our project guide “Mrs. M.PRASADA RAO”, KHIT, Guntur, for her excellent guidance right from selection of the project and her valuable suggestions throughout this project work.

We are thankful to all teaching and non-teaching staff of the COMPUTER ENGINEERING department and management and my friends for their direct and indirect work provided to us in completing the project.

- | | |
|-------------------|----------------|
| 1. Y.BHANU SAI | (23608-CM-064) |
| 2. G.NANDA KISHOR | (23608-CM-013) |
| 3. K.AKASH | (23608-CM-071) |
| 4. G.BHASKAR | (23608-CM-010) |

ABSTRACT

The rapid advancement of Artificial Intelligence (AI) and computer vision has opened new possibilities for creative applications that merge technology with art. One such innovation is the *Image to Sketch Application using AI*, which allows users to convert real-world images into artistic sketch representations. This project integrates **OpenCV-based image processing** with **deep learning models (ONNX)** to achieve high-quality sketch effects in both static and real-time modes.

The system provides multiple modes of operation, including **image upload, camera capture, and live sketching through CameraX**. Uploaded or captured images undergo processing pipelines that include **grayscale conversion, edge detection, noise reduction, and AI-based stylization** to generate sketch-like visuals. In live mode, frames are continuously captured, analyzed, and transformed into sketches in real-time, providing an interactive experience similar to augmented reality filters.

The application is developed in **Android Studio using Kotlin and Java**, ensuring cross-device compatibility and efficient performance on mobile devices. The **proposed system** overcomes the limitations of traditional manual sketching and basic filter-based applications by leveraging AI models for accuracy, speed, and enhanced realism. Furthermore, the app includes user-friendly features such as saving processed images, live previews, and multiple sketching styles.

This work demonstrates the growing synergy between **AI, mobile development, and computer vision**, providing a practical solution for entertainment, education, and design purposes. The project not only improves accessibility to artistic tools but also highlights the potential of embedding AI-based creative applications directly into mobile platforms.

INDEX

CHAPTER NO	NAME OF THE CONTENT	PAGE.NO
1	INTRODUCTION	2
2	LITERATURE SURVEY	3
	2.1 Existing System	4
	2.2 Proposed System	5
3	PROPOSED OF WORK	6
	3.1 Problem Statement	7
	3.2 Objectives	7
	3.3 Methodology	8
4	SYSTEM REQUIREMENTS SPECIFICATION	11
	4.1 Hardware Requirements	12
	4.2 Software Requirements	13
5	SYSTEM DESIGN	14
	5.1 UML Diagram	15
	5.1.1 Use Case Diagram	16
	5.1.2 Class Diagram	17
	5.1.3 Activity Diagram	18
	5.1.4 Sequence Diagram	19
6	IMPLEMENTATION	20
	6.1 Coding	21
7	SCREENSHOTS	80
8	CONCLUSION	87
9	FUTURE SCOPE	89
10	REFERENCES	91

LIST OF FIGURES

FIG.NO	NAME OF THE FIGURE	PAGE NO
5.1.1	Use Case Diagram	16
5.1.2	Class Diagram	17
5.1.3	Activity Diagram	18
5.1.4	Sequence Diagram	19
7.1	Home Page	80
7.2.1	OpenCV Mode	81
7.2.1.1	Saved Image	81
7.2.2	ONNX Mode	82
7.2.2.1	Saved Image	82
7.2.3	Stylize Mode(optional)	82
7.2.3.1	Saved Image	82
7.3.1	Capture Screen	83
7.3.2	Capture Mode	84
7.3.3	Saved Image	84
7.4.1	Live Mode	85
7.4.2	Live Preview	85

CHAPTER-1

INTRODUCTION

1.INTRODUCTION

The rapid advancements in **Artificial Intelligence (AI)** and **Computer Vision** have enabled innovative applications in multimedia, art, and entertainment. Among these, *image-to-sketch conversion* has gained significant attention due to its ability to transform ordinary images into artistic illustrations. Traditional sketch conversion relies on **edge detection and filtering techniques**, whereas modern approaches utilize **deep learning models** for producing more realistic and detailed sketches.

This project, **Image to Sketch Application using AI**, is developed in **Android Studio** using **Java/Kotlin** with integration of **OpenCV** and **ONNX models**. The application offers multiple modes of operation:

- **Upload Mode** – users can select images from the gallery.
- **Capture Mode** – images are captured directly using the camera.
- **Live Mode** – real-time processing applies sketch effects while streaming from the camera.

Additionally, the application provides options for different processing techniques:

1. **OpenCV-based sketching** (edge detection & filtering).
2. **ONNX model-based sketch generation** (AI-driven).
3. **Stylization effects** for artistic transformations.

The proposed system overcomes the limitations of existing manual sketching or static applications by enabling **real-time sketch generation, portability on mobile devices, and AI-driven accuracy**. The project not only serves as an artistic tool but also demonstrates the application of **AI in mobile vision systems**, making it suitable for academic, creative, and entertainment use cases.

CHAPTER-2

LITERATURE SURVEY

2.LITERATURE SURVEY

2.1 EXISTING SYSTEM

In the existing systems, most image-to-sketch applications rely on basic image processing techniques such as grayscale conversion, edge detection (Canny, Sobel), thresholding, and blurring. These techniques can quickly generate a pencil-sketch-like effect from an image.

Current image-to-sketch systems mainly rely on simple image processing methods like **grayscale** or **edge detection**. They are easy to implement but have major drawbacks:

- Only support **static uploads**, no real-time camera effects.
- **Low accuracy** – sketches often look artificial and lack detail.
- **Limited filters** with no advanced styles.
- **No AI integration**, so results are not realistic.
- Some require **internet/server processing**, causing delays.

2.1.2 Limitations of the Existing System:

- Output quality is not very realistic and often depends heavily on lighting and image quality.
- Fixed filters and parameters make the system less adaptive to different types of images.
- Limited support for live camera processing, since filters may not handle real-time performance smoothly.
- No integration of artificial intelligence or learning-based models, so the results lack artistic richness.

Thus, the existing system provides basic sketch conversion but fails to deliver high-quality, flexible, and real-time solutions for modern user needs.

2.2 PROPOSED SYSTEM

The proposed system introduces an AI-powered Image to Sketch application that combines both OpenCV (traditional computer vision) and ONNX/TensorFlow Lite (AI deep learning models) to achieve better sketch conversion.

The proposed system is an **AI-based Android application** that integrates **OpenCV** with **ONNX** models to overcome these issues.

Main features:

- **Three modes:** Upload from gallery, Capture from camera, and Live sketch mode.
- **AI-driven accuracy** with smooth edges and natural sketches.
- **Real-time processing** directly on the device, no internet needed.
- **User-friendly UI** with simple controls.
- **Lightweight and portable** for Android devices.

This makes the system a **complete, reliable, and real-time sketching solution** superior to existing approaches.

2.2.1 Key Features of Proposed System:

- Allows users to upload images, capture using the camera, or use live camera streaming.
- Provides different modes like OpenCV sketch, ONNX AI sketch, and Stylization effects.
- Uses machine learning models to produce more realistic and detailed sketch outputs.
- Optimized for Android devices to run locally without requiring internet, ensuring privacy and fast performance.
- Hides unnecessary controls during live preview for a full-screen immersive experience.
- Allows users to save processed images directly into the device gallery.

Advantages over Existing System:

- Produces higher quality and more artistic sketch results.
- Works in real-time with live camera feed.
- Adaptive and robust across different lighting and image conditions.

Provides a better user experience with multiple modes and saving options

CHAPTER-3

PROPOSED OF WORK

3.PROPOSED WORK

3.1 Problem statement

Existing image-to-sketch applications are often limited to simple edge detection techniques, lack real-time processing, and fail to produce natural-looking sketches. Many require server-based computation, which causes latency, dependency on internet connectivity, and privacy concerns.

Hence, there is a need for a mobile-based, real-time, AI-driven image-to-sketch system that can:

- Work efficiently on Android devices.
- Provide multiple modes of input (upload, capture, live).
- Generate high-quality, realistic sketches directly on the device.

3.2 Objectives

The main objectives of the project are:

1. To develop a **user-friendly Android application** for image-to-sketch conversion.
2. To support **multiple input sources**: gallery upload, camera capture, and live mode.
3. To integrate **AI-based models** (ONNX/TensorFlow Lite) for **realistic sketch generation**.
4. To ensure **real-time processing** with minimum latency.
5. To design a **lightweight application** that works offline and is optimized for mobile hardware.
6. To provide options for **saving and sharing** the generated sketches.

3.3 Methodology

The development of the proposed system follows a step-by-step structured methodology to ensure efficiency, accuracy, and real-time performance. The methodology is divided into several phases:

Phase 1: Requirement Analysis

- Identify hardware resources such as Android devices with minimum RAM, storage, and GPU acceleration.
- Define software dependencies like Android Studio, OpenCV, TensorFlow Lite/ONNX Runtime, and Kotlin/Java.
- Determine the functional requirements including input modes (upload, capture, live), sketch processing, and output saving.

Phase 2: System Design

- User Interface Design: Develop a simple and intuitive UI with options for selecting input sources, switching modes, and saving results.
- Module Division: Break down the system into modules such as image acquisition, pre-processing, sketch generation, and storage.
- Data Flow: Prepare UML and ER diagrams to represent data processing and interaction between modules.

Phase 3: Image Acquisition

- Upload Mode: Select image from device gallery using Android's intent system.
- Capture Mode: Use CameraX API to capture an image through the camera.
- Live Mode: Stream live camera feed and process frames in real-time.

Phase 4: Pre-processing

- Convert input images into a standard resolution for uniformity.
- Apply normalization and resizing to make them suitable for AI model input.
- Convert images to grayscale or tensor format depending on the processing mode (OpenCV or ONNX).

Phase 5: Sketch Generation

1. OpenCV Processing
 - Apply classical techniques like Canny edge detection, Gaussian blur, and Laplacian operators.
 - Enhance contrast and edges for a sketch-like output.
2. AI-based Processing (ONNX/TensorFlow Lite)
 - Use pre-trained deep learning models optimized for sketch and style transfer.
 - Convert captured or uploaded images into realistic hand-drawn sketch form.
 - Ensure inference is lightweight for real-time usage on mobile devices.

Phase 6: Post-Processing & Rendering

- Apply fine enhancements like contrast adjustment and edge smoothing.
- Render the processed sketch on the application interface.
- Allow users to switch between original and processed views for comparison.

Phase 7: Storage & Sharing

- Provide option to save the generated sketch directly to the gallery.
- Enable sharing via social media, email, or messaging apps.

Phase 8: Testing & Optimization

- Perform functional testing to ensure all modules work correctly.
- Conduct performance testing for processing speed, memory usage, and responsiveness.
- Apply model optimization techniques like quantization and pruning for faster execution on mobile hardware.
- Ensure compatibility across different Android versions and devices.

Phase 9: Deployment & Maintenance

- Package the application into an APK or AAB for distribution.
- Maintain updates by adding new models or improving UI and features.
- Collect feedback for continuous improvement.

CHAPTER-4

SYSTEM REQUIREMENT SPECIFICATION

4.SYSTEM REQUIREMENTS SPECIFICATIONS

The System Requirements Specification defines the technical and functional needs for designing and implementing the AI-based Image-to-Sketch Android Application. It ensures that both development and deployment environments are clearly defined for smooth execution of the project.

4.1 Hardware Requirements

Development Environment (Laptop/PC)

- **Processor:** Intel Core i3/i5 (or AMD equivalent) – ensures smooth project build and emulator support.
- **RAM:** Minimum 4 GB (8 GB recommended) – supports Android Studio and Gradle builds efficiently.
- **Storage:** At least 20 GB free space for Android SDK, Gradle caches, and project files.
- **Operating System:** Windows 10/11, Linux (Ubuntu), or macOS – compatible with Android Studio.

Target Device (Android Smartphone)

- **Operating System:** Android 8.0 (Oreo) or above – ensures compatibility with modern APIs.
- **RAM:** Minimum 3 GB – sufficient for handling live image processing.
- **Camera:** Minimum 8 MP with autofocus – required for good quality captures.
- **Storage:** Minimum 1 GB free space for storing processed images.
- **Battery:** 3000 mAh or higher – prevents fast drainage during live mode usage.

4.2 Software Requirements

- **IDE:** Android Studio (latest stable version).

Android Studio is the official **Integrated Development Environment (IDE)** for Android application development. It was developed by **Google** and is based on **IntelliJ IDEA**. Android Studio provides all the tools required to design, build, test, and debug Android apps efficiently. It supports programming languages like **Java**, **Kotlin**, and **C++**, and includes features such as:

- A **code editor** with intelligent code completion and refactoring.
- An **emulator** for testing applications on virtual Android devices.
- **Gradle-based build system** for easy project management.
- Built-in tools for **UI design**, **performance analysis**, and **version control**.
- **Programming Languages:**
 - 1) Java – core application logic (Backend).
 - 2) XML – UI design layouts (Frontend).
 - 3) Kotlin DSL – Gradle build scripting.
- **AI Frameworks & Libraries:**
 - 1) **OpenCV** – used for sketch conversion and image processing.
 - 2) **CameraX API** – manages live preview and image capture.
 - 3) **ONNX Runtime** – supports deep learning models.
 - 4) **Android Jetpack Components** – ensures lifecycle-aware and modern app design.
- **Storage Management:** Processed images are saved directly into the device gallery (no external DB).

Chapter-5

System Design

5.SYSTEM DESIGN

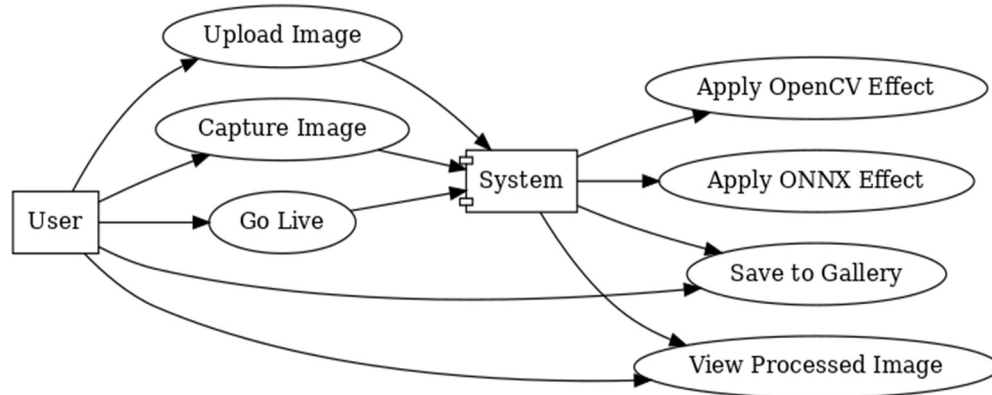
5.1 UML Diagrams

Unified Modeling Language (UML) diagrams in this project serve as structured blueprints that represent both the functional flow and system architecture of the Image-to-Sketch AI application. They visually capture how different modules such as image upload, camera capture, live sketch processing, and gallery saving interact with each other and with the user. This ensures that the transition from conceptual design to coding is clear, systematic, and free from ambiguity.

For this application, **structural diagrams** like the Class Diagram highlight the relationships among classes (e.g., MainActivity, ImageAdapter, SketchProcessor, ImageUtils). Meanwhile, **behavioral diagrams** such as Use Case, Activity, and Sequence diagrams explain how users upload an image, capture from the camera, or start live sketching, and how the system processes and outputs the sketch. These diagrams not only simplify understanding but also provide a solid foundation for implementation, debugging, and future improvements of the application.

5.1.1 Use Case Diagram

The use case Diagram of the Image to sketch APP using AI is as follows:



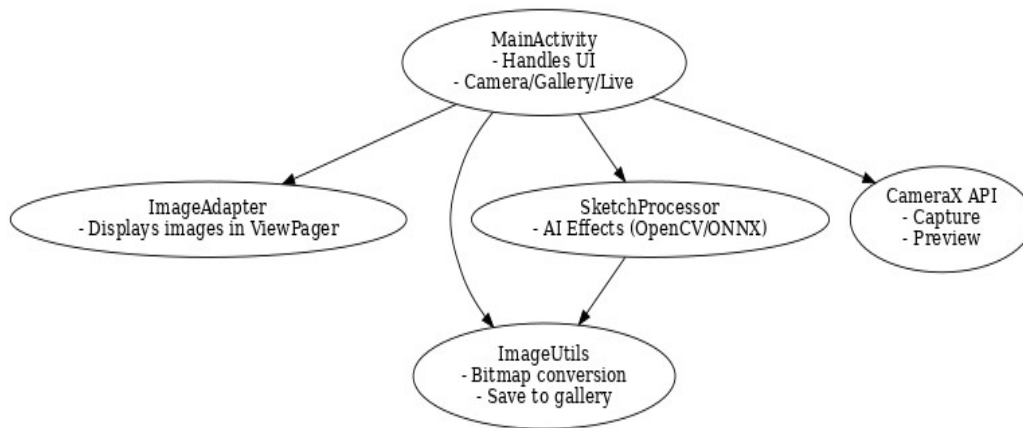
5.1 Use Case Diagram

The Use Case Diagram represents the interaction between users and the system.

- **Actors:** End User (mobile app user).
- **Use Cases:** Upload Image, Capture Image, Live Mode Processing, Apply Sketch Effect (ONNX/OpenCV), Save Image, View Gallery.
- **Purpose:** Shows how the user can interact with different features of the sketch application.
- **Understanding:** This diagram highlights the functional requirements and gives a high-level overview of what the application provides.

5.1.2 Class Diagram

The Class Diagram of the Image to sketch APP using AI is as follows:



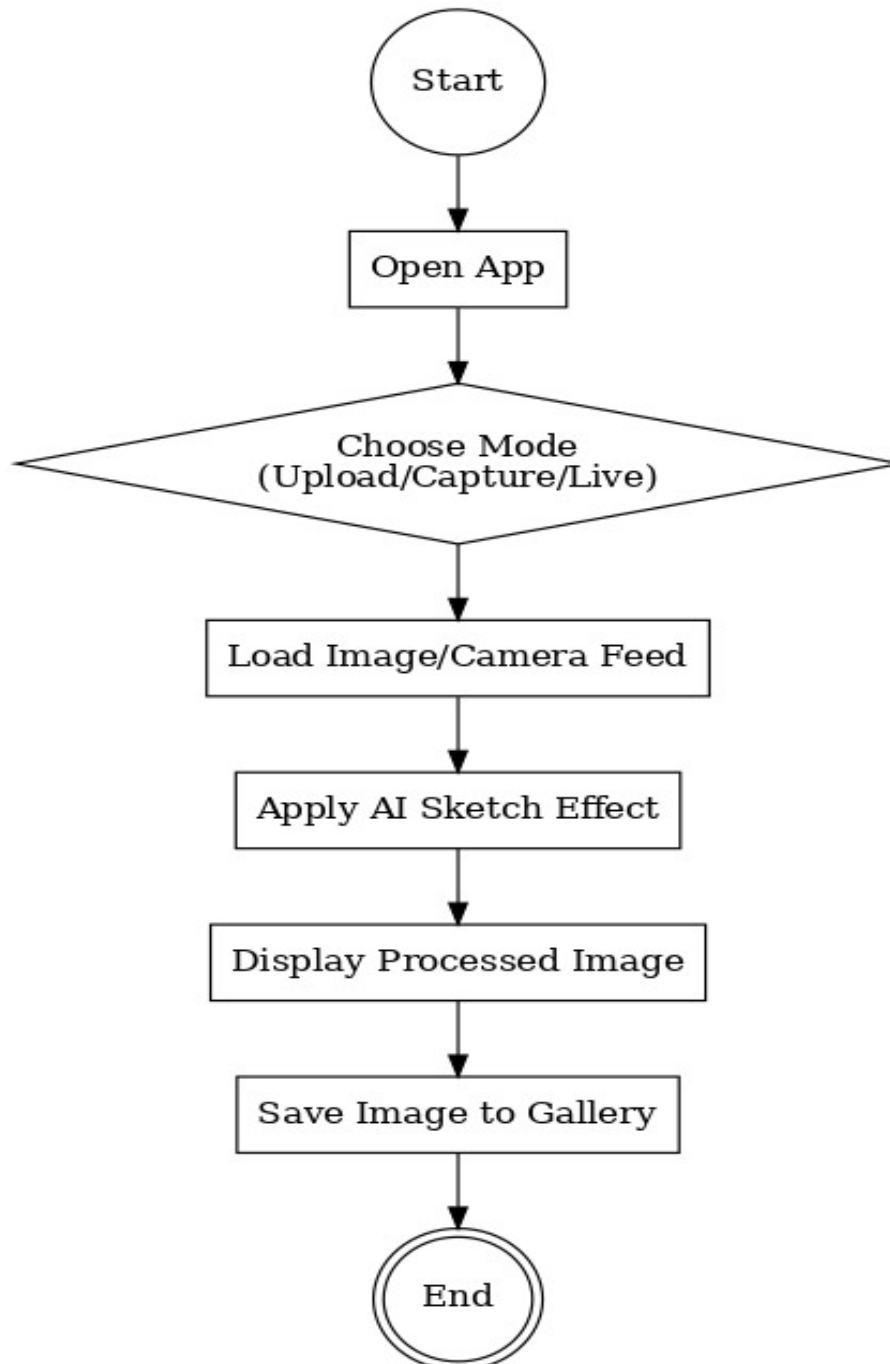
5.1.2 Class Diagram

The Class Diagram describes the structure of the application in terms of classes, attributes, and methods.

- **Main Classes:**
 - MainActivity → Manages UI and workflow.
 - ImageAdapter → Handles multiple images for display.
 - SketchProcessor → Applies AI/ML (ONNX or OpenCV) sketch effect.
 - ImageUtils → Provides helper methods for image conversion, saving, etc.
- **Relationships:**
 - MainActivity uses ImageAdapter, SketchProcessor, and ImageUtils.
- **Purpose:** Defines the system architecture and interactions between classes.
- **Understanding:** Ensures a well-structured and maintainable codebase.

5.1.3 Activity Diagram

The Activity Diagram of the Image to sketch APP using AI is as follows:



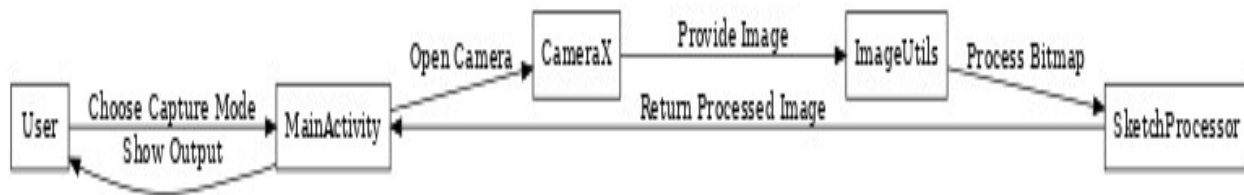
5.1.3 Activity Diagram

The Activity Diagram explains the workflow of the system in sequential steps.

- **Flow:** User opens app → selects mode (upload, capture, live) → image is processed using AI models → output is displayed → user can save image.
- **Decision Points:** Mode selection (Upload, Capture, or Live), Effect type (ONNX/OpenCV).
- **Purpose:** To visualize the control flow of different processes in the application.
- **Understanding:** It clarifies how the app behaves in various scenarios and ensures no steps are missed in implementation.

5.1.4 Sequence Diagram

The Activity Diagram of the Image to sketch APP using AI is as follows:



5.1.4 Sequence Diagram

The Sequence Diagram illustrates how objects in the system interact over time.

- **Actors and Objects:** User → MainActivity → Camera Module → ONNX/OpenCV Processor → Image Adapter → Gallery.
- **Process:**
 1. User requests an action (upload, capture, or live).
 2. MainActivity communicates with camera or gallery.
 3. Processor applies AI-based sketch effect.
 4. Processed image is returned and displayed.
 5. User can save it to the gallery.
- **Purpose:** Shows step-by-step communication and order of operations.
- **Understanding:** Helps developers ensure modules interact correctly and maintain smooth data flow.

CHAPTER-6

IMPLEMENTATION

6.IMPLEMENTATION

6.1 Coding

The implementation of the project is carried out using Java, XML, and Kotlin (for DSL integration) in Android Studio. The coding process involves modularizing the project into separate classes for handling different functionalities such as image processing, gallery management, and sketch transformations.

Main Modules of Code

1. MainActivity.java

- Controls the flow of the application.
- Handles image capture, live mode, and upload.
- Integrates OpenCV and ONNX models for sketch conversion.

2. ImageAdapter.java

- Manages image previews inside the ViewPager.
- Displays the original image and the processed sketch image side by side.

3. ImageUtils.java

- Contains utility functions for:
 - Converting ImageProxy from CameraX into Bitmap.
 - Rotating and formatting images.
 - Saving processed images into the Gallery.

4. SketchPagerAdapter.java

- Handles swipe-based navigation between different sketch filters.
- Manages layout inflating for processed sketches.

5. SketchProcessor.java

- Core AI module that applies OpenCV edge detection or ONNX-based AI models to convert images into sketch form.

6. XML Layouts

- Designed with ConstraintLayout for responsiveness.
- Includes PreviewView, ImageView, ViewPager, and interactive buttons (Upload, Capture, Live, Save, etc.).

6.2 Database

This project does not require a traditional database. Instead:

- Captured/processed images are stored directly in the device gallery.
- For live processing, images are handled in-memory using Bitmap objects.
- No external database (like SQLite or Firebase) is used, making the app lightweight and efficient.

MainActivity.java:

```
package com.example.ribhance;

import android.Manifest;

import android.content.ContentValues;

import android.content.Intent;

import android.content.pm.PackageManager;

import android.graphics.Bitmap;

import android.net.Uri;

import android.os.Build;

import android.os.Bundle;

import android.provider.MediaStore;

import android.util.Log;

import android.view.View;

import android.widget.Button;

import android.widget.LinearLayout;

import android.widget.RadioButton;

import android.widget.RadioGroup;

import android.widget.Toast;

import android.widget.ImageView;

import androidx.activity.result.ActivityResultLauncher;

import androidx.activity.result.contract.ActivityResultContracts;

import androidx.annotation.NonNull;
```

```
import androidx.appcompat.app.AppCompatActivity;

import androidx.camera.core.CameraSelector;

import androidx.camera.core.ImageAnalysis;

import androidx.camera.core.ImageCapture;

import androidx.camera.core.ImageCaptureException;

import androidx.camera.core.ImageProxy;

import androidx.camera.core.Preview;

import androidx.camera.lifecycle.ProcessCameraProvider;

import androidx.camera.view.PreviewView;

import androidx.core.app.ActivityCompat;

import androidx.core.content.ContextCompat;

import androidx.viewpager2.widget.ViewPager2;

import com.google.common.util.concurrent.ListenableFuture;

import org.opencv.android.OpenCVLoader;

import org.opencv.core.CvType;

import org.opencv.core.Mat;

import java.io.IOException;

import java.io.OutputStream;

import java.util.concurrent.ExecutorService;

import java.util.concurrent.Executors;
```

```

public class MainActivity extends AppCompatActivity {

    private static final String TAG = "MainActivity";

    private RadioGroup modeGroup, inputSourceGroup;

    private Button btnChoose, btnSave, btnStop, btnCapture;

    private LinearLayout buttonPanel;

    private ViewPager2 viewPager;

    private ImageView liveView;

    private PreviewView cameraPreview;

    private String currentMode = "OpenCV";

    private Bitmap processedBitmap, originalBitmap;

    private ImageAdapter adapter;

    private ExecutorService cameraExecutor;

    private ImageCapture imageCapture;

    private ProcessCameraProvider cameraProvider;

    private boolean liveActive = false;

    private final ActivityResultLauncher<Intent> galleryLauncher =
        registerForActivityResult(new ActivityResultContracts.StartActivityForResult(), result ->
    {

        if (result.getResultCode() == RESULT_OK && result.getData() != null) {

            Uri imageUri = result.getData().getData();

            try {

                originalBitmap =
MediaStore.Images.Media.getBitmap(this.getContentResolver(), imageUri);

```



```

        processAndDisplay();

    } catch (IOException e) {

        Log.e(TAG, "Gallery load failed: " + e.getMessage());

        Toast.makeText(this, "Failed to load image", Toast.LENGTH_SHORT).show();

    }

}

});

// Load OpenCV

static {

    if (!OpenCVLoader.initDebug()) {

        Log.e("OpenCV", "OpenCV not loaded!");

    } else {

        Log.d("OpenCV", "OpenCV loaded successfully!");

    }

}

@Override

protected void onCreate(Bundle savedInstanceState) {

    super.onCreate(savedInstanceState);

    setContentView(R.layout.activity_main);

    // test OpenCV quickly (optional)

    try {

        Mat testMat = new Mat(1, 1, CvType.CV_8UC1);

```

```

        Log.d(TAG, "OpenCV Mat ok: " + testMat);
    } catch (Exception e) {

        Log.e(TAG, "OpenCV test failed: " + e.getMessage());
    }

    SketchProcessor.initOnnx(this); // attempt to load ONNX model

    // find views

    modeGroup = findViewById(R.id.modeGroup);

    inputSourceGroup = findViewById(R.id.inputSourceGroup);

    btnChoose = findViewById(R.id.btnChoose);

    btnSave = findViewById(R.id.btnSave);

    btnStop = findViewById(R.id.btnStop);

    btnCapture = findViewById(R.id.btnCapture);

    buttonPanel = findViewById(R.id.buttonPanel);

    viewPager = findViewById(R.id.viewPager);

    liveView = findViewById(R.id.liveView);

    cameraPreview = findViewById(R.id.cameraPreview);

    adapter = new ImageAdapter();

    viewPager.setAdapter(adapter);

    cameraExecutor = Executors.newSingleThreadExecutor();

    // Mode selection listener

    modeGroup.setOnCheckedChangeListener((group, checkedId) -> {

        RadioButton rb = findViewById(checkedId);

```

```

        if (rb != null) {

            currentMode = rb.getText().toString();

            Log.d(TAG, "Mode selected: " + currentMode);

        }

    });

    // Choose button (handles upload / capture binding / live start)

    btnChoose.setOnClickListener(v -> {

        int checked = inputSourceGroup.getCheckedRadioButtonId();

        if (checked == R.id.radioUpload) {

            // launch gallery

            Intent intent = new Intent(Intent.ACTION_PICK,
MediaStore.Images.Media.EXTERNAL_CONTENT_URI);

            galleryLauncher.launch(intent);

        } else if (checked == R.id.radioCapture) {

            startCameraForCapture();

        } else if (checked == R.id.radioLive) {

            startLiveCamera();

        } else {

            Toast.makeText(this, "Select a source first", Toast.LENGTH_SHORT).show();

        }

    });

    // Save processed image

    btnSave.setOnClickListener(v -> {

```

```

        if (processedBitmap == null) {

            Toast.makeText(this, "No processed image to save", Toast.LENGTH_SHORT).show();

            return;

        }

        Uri uri = ImageUtils.saveBitmapToGallery(this, processedBitmap);

        if (uri != null) {

            Toast.makeText(this, "Saved to Gallery", Toast.LENGTH_SHORT).show();

        } else {

            Toast.makeText(this, "Save failed", Toast.LENGTH_SHORT).show();

        }

    });

    // Stop live

    btnStop.setOnClickListener(v -> stopLiveCamera());

    // Capture snapshot when preview bound

    btnCapture.setOnClickListener(v -> captureImage());

}

// ----- Capture Mode binding -----

private void startCameraForCapture() {

    if (!allPermissionsGranted()) {

        ActivityCompat.requestPermissions(this, new String[]{Manifest.permission.CAMERA},
101);

        return;

    }

```

```

    ListenableFuture<ProcessCameraProvider> cameraProviderFuture =
        ProcessCameraProvider.getInstance(this);

    cameraProviderFuture.addListener(() -> {

        try {

            cameraProvider = cameraProviderFuture.get();

            CameraSelector cameraSelector = new CameraSelector.Builder()

                .requireLensFacing(CameraSelector.LENS_FACING_BACK).build();

            imageCapture = new ImageCapture.Builder().build();

            Preview preview = new Preview.Builder().build();

            preview.setSurfaceProvider(cameraPreview.getSurfaceProvider());

            cameraProvider.unbindAll();

            cameraProvider.bindToLifecycle(this, cameraSelector, preview, imageCapture);

            // Show preview + capture button; hide other UI to avoid overlap

            cameraPreview.setVisibility(View.VISIBLE);

            liveView.setVisibility(View.GONE);

            viewPager.setVisibility(View.GONE);

            buttonPanel.setVisibility(View.VISIBLE);

            btnCapture.setVisibility(View.VISIBLE);

            btnChoose.setVisibility(View.GONE);

            btnSave.setVisibility(View.GONE);

            btnStop.setVisibility(View.GONE);

        } catch (Exception e) {

            Log.e(TAG, "startCameraForCapture error: " + e.getMessage());

```

```

        Toast.makeText(this, "Camera failed to start for capture",
Toast.LENGTH_SHORT).show();

    }

    }, ContextCompat.getMainExecutor(this));

}

private void captureImage() {

    if (imageCapture == null) {

        Toast.makeText(this, "Camera not ready", Toast.LENGTH_SHORT).show();

        return;

    }

    imageCapture.takePicture(ContextCompat.getMainExecutor(this),

        new ImageCapture.OnImageCapturedCallback() {

            @Override

            public void onCaptureSuccess(@NonNull ImageProxy image) {

                Bitmap bmp = ImageUtils.imageProxyToBitmap(image);

                image.close();

                if (bmp != null) {

                    originalBitmap = bmp;

                    processAndDisplay();

                } else {

                    Toast.makeText(MainActivity.this, "Capture returned empty bitmap",
Toast.LENGTH_SHORT).show();

                }

            }

        }

    );
}

```

```

        // restore UI

        btnCapture.setVisibility(View.GONE);

        btnChoose.setVisibility(View.VISIBLE);

        btnSave.setVisibility(View.VISIBLE);

    }

    @Override

    public void onError(@NonNull ImageCaptureException exception) {

        Log.e(TAG, "captureImage error: " + exception.getMessage());

        Toast.makeText(MainActivity.this, "Capture failed",
Toast.LENGTH_SHORT).show();

    }

});

}

// ----- Live Camera with Filters -----

private void startLiveCamera() {

    if (!allPermissionsGranted()) {

        ActivityCompat.requestPermissions(this, new String[]{Manifest.permission.CAMERA},
101);

        return;

    }

    ListenableFuture<ProcessCameraProvider> cameraProviderFuture =
ProcessCameraProvider.getInstance(this);

    cameraProviderFuture.addListener(() -> {

        try {

```

```

cameraProvider = cameraProviderFuture.get();

CameraSelector cameraSelector = new CameraSelector.Builder()

    .requireLensFacing(CameraSelector.LENS_FACING_BACK).build();

ImageAnalysis analysis = new ImageAnalysis.Builder()

    .setBackpressureStrategy(ImageAnalysis.STRATEGY_KEEP_ONLY_LATEST)

    .build();

liveActive = true;

final int[] frameCounter = {0};

analysis.setAnalyzer(cameraExecutor, image -> {

    if (!liveActive) {

        image.close();

        return;

    }

    // throttle ONNX live processing to every 3rd frame to reduce CPU (safe fallback)

    frameCounter[0]++;

    boolean doProcess = true;

    if ("ONNX".equalsIgnoreCase(currentMode)) {

        // process fewer frames for heavy ONNX

        doProcess = (frameCounter[0] % 3 == 0);

    }

    Bitmap bmp = ImageUtils.imageProxyToBitmap(image);

    image.close();

```



```

        if (bmp == null) return;

        if (doProcess) {

            Bitmap processed = SketchProcessor.applyEffect(bmp, currentMode);

            runOnUiThread() -> {

                // show processed live

                liveView.setVisibility(View.VISIBLE);

                cameraPreview.setVisibility(View.GONE);

                viewPager.setVisibility(View.GONE);

                liveView.setImageBitmap(processed);

            });

        } else {

            // optionally show a quick preview (raw) - we keep processing off-frame to avoid
flicker

        }

    });

    cameraProvider.unbindAll();

    cameraProvider.bindToLifecycle(this, cameraSelector, analysis);


    // hide panels and only show stop

    buttonPanel.setVisibility(View.GONE);

    btnStop.setVisibility(View.VISIBLE);

    Toast.makeText(this, "Live camera started", Toast.LENGTH_SHORT).show();

} catch (Exception e) {

```

```

        Log.e(TAG, "startLiveCamera error: " + e.getMessage());

        Toast.makeText(this, "Live camera failed to start", Toast.LENGTH_SHORT).show();

    }

}, ContextCompat.getMainExecutor(this));
}

private void stopLiveCamera() {

    liveActive = false;

    if (cameraProvider != null) {

        cameraProvider.unbindAll();

    }

    liveView.setVisibility(View.GONE);

    cameraPreview.setVisibility(View.GONE);

    viewPager.setVisibility(View.VISIBLE);

    // restore panel

    buttonPanel.setVisibility(View.VISIBLE);

    btnStop.setVisibility(View.GONE);

    Toast.makeText(this, "Live stopped", Toast.LENGTH_SHORT).show();

}

// ----- Process and Display (Upload or Capture result) -----

private void processAndDisplay() {

    if (originalBitmap == null) {

        Toast.makeText(this, "No image to process", Toast.LENGTH_SHORT).show();
    }
}

```

```

        return;
    }

    // If user selected ONNX but ONNX failed to load, fallback to OpenCV with a message
    if ("ONNX".equalsIgnoreCase(currentMode) && !SketchProcessor.isOnnxLoaded()) {
        Toast.makeText(this, "ONNX model not loaded — falling back to OpenCV",
            Toast.LENGTH_SHORT).show();

        currentMode = "OpenCV";
    }

    processedBitmap = SketchProcessor.applyEffect(originalBitmap, currentMode);
    adapter.setBitmaps(originalBitmap, processedBitmap);

    // show ViewPager (which displays both images side-by-side)
    viewPager.setVisibility(View.VISIBLE);
    liveView.setVisibility(View.GONE);
    cameraPreview.setVisibility(View.GONE);

    // ensure UI buttons restored
    buttonPanel.setVisibility(View.VISIBLE);
    btnStop.setVisibility(View.GONE);
    btnCapture.setVisibility(View.GONE);
    btnChoose.setVisibility(View.VISIBLE);
    btnSave.setVisibility(View.VISIBLE);
}

// ----- Save Image -----
private void saveImage() {

```

```

        if (processedBitmap == null) {

            Toast.makeText(this, "No processed image to save", Toast.LENGTH_SHORT).show();

            return;

        }

        Uri uri = ImageUtils.saveBitmapToGallery(this, processedBitmap);

        if (uri != null) {

            Toast.makeText(this, "Saved to Gallery", Toast.LENGTH_SHORT).show();

        } else {

            Toast.makeText(this, "Save failed", Toast.LENGTH_SHORT).show();

        }

    }

    private boolean allPermissionsGranted() {

        return ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA) ==
        PackageManager.PERMISSION_GRANTED;

    }

    @Override

    protected void onDestroy() {

        super.onDestroy();

        if (cameraExecutor != null) cameraExecutor.shutdown();

    }

}

```

SketchProcessor.java:

```
package com.example.ribhance;

import android.content.Context;

import android.graphics.Bitmap;

import android.graphics.Color;

import android.util.Log;

import org.opencv.android.Utls;

import org.opencv.core.Mat;

import org.opencv.core.Size;

import org.opencv.core.Core;

import org.opencv.imgproc.Imgproc;

import org.opencv.photo.Photo;

import java.io.File;

import java.io.FileOutputStream;

import java.io.InputStream;

import java.util.Collections;

import ai.onnxruntime.OnnxTensor;

import ai.onnxruntime.OrtEnvironment;

import ai.onnxruntime.OrtSession;

public class SketchProcessor {

    private static final String TAG = "SketchProcessor";
```

```

private static OrtEnvironment env;

private static OrtSession onnxSession;

public static void initOnnx(Context context) {

    try {

        if (env == null) env = OrtEnvironment.getEnvironment();

        if (onnxSession == null) {

            String modelPath = copyAssetToFile(context, "sketch.onnx");

            if (modelPath != null) {

                onnxSession = env.createSession(modelPath, new OrtSession.SessionOptions());

                Log.d(TAG, "ONNX Model loaded successfully!");

            } else {

                Log.w(TAG, "sketch.onnx not found in assets");

            }

        }

    } catch (Exception e) {

        Log.e(TAG, "Failed to load ONNX model: " + e.getMessage());

        onnxSession = null;

    }

}

public static boolean isOnnxLoaded() {

    return onnxSession != null;
}

```

```

public static Bitmap applyEffect(Bitmap input, String mode) {

    if (input == null) return null;

    try {

        switch (mode) {

            case "OpenCV":

                return applyOpenCVSketch(input);

            case "ONNX":

                return applyOnnxSketch(input);

            case "Stylize":

                return applyStylizeEffect(input);

            default:

                return input;

        }

    } catch (Exception e) {

        Log.e(TAG, "applyEffect error: " + e.getMessage());

        return input;

    }

}

// OpenCV pencil sketch

private static Bitmap applyOpenCVSketch(Bitmap input) {

```

```

Mat src = new Mat();

try {

    Utils.bitmapToMat(input, src);

    if (src.channels() == 4) Imgproc.cvtColor(src, src, Imgproc.COLOR_RGBA2BGR);

    Mat gray = new Mat();

    Imgproc.cvtColor(src, gray, Imgproc.COLOR_BGR2GRAY);

    Mat inverted = new Mat();

    Core.bitwise_not(gray, inverted);

    Mat blur = new Mat();

    Imgproc.GaussianBlur(inverted, blur, new Size(21, 21), 0);

    Mat invertedBlur = new Mat();

    Core.bitwise_not(blur, invertedBlur);

    Mat sketch = new Mat();

    Core.divide(gray, invertedBlur, sketch, 256.0);

    Mat sketchRGBA = new Mat();

    Imgproc.cvtColor(sketch, sketchRGBA, Imgproc.COLOR_GRAY2RGBA);

    Bitmap output = Bitmap.createBitmap(sketchRGBA.cols(), sketchRGBA.rows(),
    Bitmap.Config.ARGB_8888);

    Utils.matToBitmap(sketchRGBA, output);

    // release

    src.release(); gray.release(); inverted.release(); blur.release(); invertedBlur.release();
    sketch.release(); sketchRGBA.release();

```



```

        return output;

    } catch (Exception e) {

        Log.e(TAG, "applyOpenCVSketch error: " + e.getMessage());

        if (src != null) src.release();

        return input;

    }

}

// ONNX model inference (safe handling)

private static Bitmap applyOnnxSketch(Bitmap input) {

    if (onnxSession == null) {

        Log.w(TAG, "ONNX session null, falling back to OpenCV");

        return applyOpenCVSketch(input);

    }

    try {

        final int modelSize = 256;

        Bitmap resized = Bitmap.createScaledBitmap(input, modelSize, modelSize, true);

        float[][][][] inputData = new float[1][3][modelSize][modelSize];

        for (int y = 0; y < modelSize; y++) {

            for (int x = 0; x < modelSize; x++) {

                int px = resized.getPixel(x, y);

                inputData[0][0][y][x] = ((px >> 16) & 0xFF) / 255.0f;
            }
        }
    }
}

```

```

        inputData[0][1][y][x] = ((px >> 8) & 0xFF) / 255.0f;

        inputData[0][2][y][x] = (px & 0xFF) / 255.0f;

    }

}

OnnxTensor tensor = OnnxTensor.createTensor(env, inputData);

OrtSession.Result result = onnxSession.run(

    Collections.singletonMap(onnxSession.getInputNames().iterator().next(), tensor)

);

Object rawOut = result.get(0).getValue();

Bitmap outBmp = null;

if (rawOut instanceof float[][][]) {

    float[][][] out = (float[][][]) rawOut;

    int h = out[0][0].length;

    int w = out[0][0][0].length;

    outBmp = Bitmap.createBitmap(w, h, Bitmap.Config.ARGB_8888);

    for (int y = 0; y < h; y++) {

        for (int x = 0; x < w; x++) {

            int r = Math.min(255, Math.max(0, (int) (out[0][0][y][x] * 255)));

            int g = Math.min(255, Math.max(0, (int) (out[0][1][y][x] * 255)));

            int b = Math.min(255, Math.max(0, (int) (out[0][2][y][x] * 255)));

            outBmp.setPixel(x, y, Color.rgb(r, g, b));

```

```

        }

    }

    Bitmap scaled = Bitmap.createScaledBitmap(outBmp, input.getWidth(),
input.getHeight(), true);

    outBmp.recycle();

    tensor.close();

    result.close();

    return scaled;

} else {

    Log.e(TAG, "Unexpected ONNX output type: " + rawOut.getClass());

    tensor.close();

    result.close();

    return applyOpenCVSketch(input);

}

} catch (Exception e) {

    Log.e(TAG, "applyOnnxSketch error: " + e.getMessage());

    return applyOpenCVSketch(input);

}

}

// Stylize (cartoon)

private static Bitmap applyStylizeEffect(Bitmap input) {

    Mat src = new Mat();

```

```

try {

    Utils.bitmapToMat(input, src);

    if (src.channels() == 4) Imgproc.cvtColor(src, src, Imgproc.COLOR_RGBA2BGR);

    Mat dst = new Mat();

    Photo.stylization(src, dst, 60f, 0.45f);

    Mat dstRGBA = new Mat();

    Imgproc.cvtColor(dst, dstRGBA, Imgproc.COLOR_BGR2RGBA);

    Bitmap output = Bitmap.createBitmap(dstRGBA.cols(), dstRGBA.rows(),
    Bitmap.Config.ARGB_8888);

    Utils.matToBitmap(dstRGBA, output);

    src.release(); dst.release(); dstRGBA.release();

    return output;

} catch (Exception e) {

    Log.e(TAG, "applyStylizeEffect error: " + e.getMessage());

    if (src != null) src.release();

    return input;

}

}

// copy asset helper

private static String copyAssetToFile(Context context, String assetName) {

    try {

        File outFile = new File(context.getFilesDir(), assetName);

```

```
if (!outFile.exists()) {  
    InputStream in = context.getAssets().open(assetName);  
    FileOutputStream out = new FileOutputStream(outFile);  
    byte[] buffer = new byte[1024];  
    int read;  
    while ((read = in.read(buffer)) != -1) out.write(buffer, 0, read);  
    in.close(); out.close();  
}  
return outFile.getAbsolutePath();  
} catch (Exception e) {  
    Log.e(TAG, "copyAssetToFile error: " + e.getMessage());  
    return null;  
}  
}  
}
```

SketchPagerAdapter.java:

```
package com.example.ribhance;

import android.graphics.Bitmap;

import android.view.LayoutInflater;

import android.view.View;

import android.view.ViewGroup;

import android.widget.ImageView;

import androidx.annotation.NonNull;

import androidx.recyclerview.widget.RecyclerView;

import java.util.ArrayList;

import java.util.List;

public class SketchPagerAdapter extends
RecyclerView.Adapter<SketchPagerAdapter.ViewHolder> {

    private final List<Bitmap> bitmaps = new ArrayList<>();

    public void setBitmaps(Bitmap original, Bitmap sketch) {

        bitmaps.clear();

        bitmaps.add(original);

        bitmaps.add(sketch);

        notifyDataSetChanged();

    }

    @NonNull

    @Override
```

```

public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {

    ImageView imageView = (ImageView) LayoutInflater.from(parent.getContext())

        .inflate(R.layout.item_page, parent, false);

    return new ViewHolder(imageView);

}

@Override

public void onBindViewHolder(@NonNull ViewHolder holder, int position) {

    holder.imageView.setImageBitmap(bitmaps.get(position));

}

@Override

public int getItemCount() {

    return bitmaps.size();

}

static class ViewHolder extends RecyclerView.ViewHolder {

    ImageView imageView;

    ViewHolder(@NonNull View itemView) {

        super(itemView);

        imageView = (ImageView) itemView;

    }

}

}

```

ImageAdapter.java:

```
package com.example.ribhance;

import android.graphics.Bitmap;

import android.view.View;

import android.view.ViewGroup;

import android.widget.ImageView;

import android.widget.LinearLayout;

import androidx.annotation.NonNull;

import androidx.recyclerview.widget.RecyclerView;

public class ImageAdapter extends RecyclerView.Adapter<ImageAdapter.VH> {

    private Bitmap original, processed;

    public void setBitmaps(Bitmap orig, Bitmap proc) {

        this.original = orig;

        this.processed = proc;

        notifyDataSetChanged();

    }

    @NonNull

    @Override

    public VH onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {

        LinearLayout container = new LinearLayout(parent.getContext());

        container.setOrientation(LinearLayout.HORIZONTAL);
```



```

        container.setLayoutParams(new
        ViewGroup.LayoutParams(ViewGroup.LayoutParams.MATCH_PARENT,
        ViewGroup.LayoutParams.MATCH_PARENT));

        ImageView iv1 = new ImageView(parent.getContext());

        ImageView iv2 = new ImageView(parent.getContext());

        LinearLayout.LayoutParams lp = new LinearLayout.LayoutParams(0,
        ViewGroup.LayoutParams.MATCH_PARENT, 1f);

        iv1.setLayoutParams(lp);

        iv2.setLayoutParams(lp);

        iv1.setScaleType(ImageView.ScaleType.FIT_CENTER);

        iv2.setScaleType(ImageView.ScaleType.FIT_CENTER);

        container.addView(iv1);

        container.addView(iv2);

        return new VH(container, iv1, iv2);
    }

    @Override

    public void onBindViewHolder(@NonNull VH holder, int position) {

        holder.left.setImageBitmap(original);

        holder.right.setImageBitmap(processed);

    }

    @Override

    public int getItemCount() {

```

```

        // single page showing both images side-by-side

```

```
        return l;  
    }  
  
    static class VH extends RecyclerView.ViewHolder {  
  
        ImageView left, right;  
  
        VH(@NonNull View itemView, ImageView l, ImageView r) {  
  
            super(itemView);  
  
            left = l;  
  
            right = r;  
  
        }  
    }  
}
```

ImageUtils.java:

```
package com.example.ribhance;

import android.content.ContentValues;

import android.content.Context;

import android.graphics.Bitmap;

import android.graphics.BitmapFactory;

import android.graphics.ImageFormat;

import android.graphics.Matrix;

import android.graphics.Rect;

import android.graphics.YuvImage;

import android.net.Uri;

import android.os.Build;

import android.provider.MediaStore;

import android.util.Log;

import androidx.camera.core.ImageProxy;

import java.io.ByteArrayOutputStream;

import java.io.OutputStream;

import java.nio.ByteBuffer;

public class ImageUtils {

    private static final String TAG = "ImageUtils";

    // Convert ImageProxy (YUV_420_888) -> Bitmap (correct rotation)
```

```

public static Bitmap imageProxyToBitmap(ImageProxy image) {
    try {
        if (image == null) return null;

        if (image.getFormat() == ImageFormat.YUV_420_888) {

            ImageProxy.PlaneProxy[] planes = image.getPlanes();

            ByteBuffer yBuffer = planes[0].getBuffer();

            ByteBuffer uBuffer = planes[1].getBuffer();

            ByteBuffer vBuffer = planes[2].getBuffer();

            int ySize = yBuffer.remaining();

            int uSize = uBuffer.remaining();

            int vSize = vBuffer.remaining();

            byte[] nv21 = new byte[ySize + uSize + vSize];

            // U and V are swapped for NV21

            yBuffer.get(nv21, 0, ySize);

            vBuffer.get(nv21, ySize, vSize);

            uBuffer.get(nv21, ySize + vSize, uSize);

            YuvImage yuvImage = new YuvImage(nv21, ImageFormat.NV21, image.getWidth(),
image.getHeight(), null);

            ByteArrayOutputStream out = new ByteArrayOutputStream();

            yuvImage.compressToJpeg(new Rect(0, 0, image.getWidth(), image.getHeight()), 90,
out);

            byte[] jpegBytes = out.toByteArray();

```

```

        Bitmap rawBitmap = BitmapFactory.decodeByteArray(jpegBytes, 0,
jpegBytes.length);

        // Rotate if needed

        Matrix matrix = new Matrix();

        int rotation = image.getImageInfo().getRotationDegrees();

        if (rotation != 0) matrix.postRotate(rotation);

        Bitmap rotated = Bitmap.createBitmap(rawBitmap, 0, 0, rawBitmap.getWidth(),
rawBitmap.getHeight(), matrix, true);

        rawBitmap.recycle();

        return rotated;

    } else {

        // fallback: decode first plane

        ByteBuffer buffer = image.getPlanes()[0].getBuffer();

        byte[] bytes = new byte[buffer.remaining()];

        buffer.get(bytes);

        return BitmapFactory.decodeByteArray(bytes, 0, bytes.length);

    }

} catch (Exception e) {

    Log.e(TAG, "imageProxyToBitmap failed: " + e.getMessage());

    return null;

}

}

```

```

// Save bitmap to gallery (Android Q+ and older fallback)

public static Uri saveBitmapToGallery(Context context, Bitmap bitmap) {

    if (bitmap == null) return null;

    Uri imageUri = null;

    OutputStream out = null;

    try {

        ContentValues values = new ContentValues();

        String displayName = "sketch_" + System.currentTimeMillis() + ".png";

        values.put(MediaStore.Images.Media.DISPLAY_NAME, displayName);

        values.put(MediaStore.Images.Media.MIME_TYPE, "image/png");

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {

            values.put(MediaStore.Images.Media.RELATIVE_PATH, "Pictures/SketchApp");

            imageUri =
context.getContentResolver().insert(MediaStore.Images.Media.EXTERNAL_CONTENT_URI,
values);

            if (imageUri != null) {

                out = context.getContentResolver().openOutputStream(imageUri);

                bitmap.compress(Bitmap.CompressFormat.PNG, 100, out);

            }

        } else {

            // pre-Android Q fallback

```

```

        String imagesDir =
MediaStore.Images.Media.insertImage(context.getContentResolver(), bitmap, displayName,
"Saved from Ribhance");

        if (imagesDir != null) {

            imageUri = Uri.parse(imagesDir);

        }

    }

    if (out != null) out.close();

    Log.d(TAG, "Saved image: " + imageUri);

    return imageUri;

} catch (Exception e) {

    Log.e(TAG, "saveBitmapToGallery failed: " + e.getMessage());

    try { if (out != null) out.close(); } catch (Exception ignored) {}

    return null;

}

}

}

```

Activity main.xml:

```
<?xml version="1.0" encoding="utf-8"?>

<androidx.constraintlayout.widget.ConstraintLayout

    xmlns:android="http://schemas.android.com/apk/res/android"

    xmlns:app="http://schemas.android.com/apk/res-auto"

    android:id="@+id/rootLayout"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    android:background="#F9F9F9">

    <!-- Title -->

    <TextView

        android:id="@+id/appTitle"

        android:layout_width="wrap_content"

        android:layout_height="wrap_content"

        android:text="🤖 Ribhance - AI Image to Sketch"

        android:textSize="20sp"

        android:textStyle="bold"

        android:textColor="#4A90E2"

        android:fontFamily="sans-serif-medium"

        app:layout_constraintTop_toTopOf="parent"

        app:layout_constraintStart_toStartOf="parent"
```



```

        app:layout_constraintEnd_toEndOf="parent"

        android:layout_marginTop="16dp"/>

<!-- Camera Preview (for Capture) -->

<androidx.camera.view.PreviewView

    android:id="@+id/cameraPreview"

    android:layout_width="0dp"

    android:layout_height="0dp"

    android:background="#000000"

    android:visibility="gone"

    app:layout_constraintTop_toBottomOf="@id/appTitle"

    app:layout_constraintStart_toStartOf="parent"

    app:layout_constraintEnd_toEndOf="parent"

    app:layout_constraintBottom_toBottomOf="parent"

    android:layout_marginTop="8dp"/>

<!-- Live View (for Live Sketch Filter) -->

<ImageView

    android:id="@+id/liveView"

    android:layout_width="0dp"

    android:layout_height="0dp"

    android:scaleType="centerCrop"

    android:visibility="gone"

```

```

        android:background="#000"

        app:layout_constraintTop_toBottomOf="@id/appTitle"

        app:layout_constraintStart_toStartOf="parent"

        app:layout_constraintEnd_toEndOf="parent"

        app:layout_constraintBottom_toBottomOf="parent"

        android:layout_marginTop="8dp"/>

<!-- ViewPager (Original + Processed side-by-side) -->

<androidx.viewpager2.widget.ViewPager2

    android:id="@+id/viewPager"

    android:layout_width="0dp"

    android:layout_height="250dp"

    android:visibility="gone"

    app:layout_constraintTop_toBottomOf="@id/appTitle"

    app:layout_constraintStart_toStartOf="parent"

    app:layout_constraintEnd_toEndOf="parent"

    android:layout_marginTop="8dp"

    android:background="#FFFFFF"

    android:elevation="4dp"/>

<!-- Mode Selection -->

<RadioGroup

    android:id="@+id/modeGroup"

```

```

    android:layout_width="0dp"

    android:layout_height="wrap_content"

    android:orientation="horizontal"

    android:gravity="center"

    app:layout_constraintTop_toBottomOf="@id/viewPager"

    app:layout_constraintStart_toStartOf="parent"

    app:layout_constraintEnd_toEndOf="parent"

    android:layout_marginTop="12dp">

<RadioButton

    android:id="@+id/radioOpenCV"

    android:layout_width="wrap_content"

    android:layout_height="wrap_content"

    android:text="OpenCV"

    android:checked="true"

    android:textColor="#000"/>

<RadioButton

    android:id="@+id/radioOnnx"

    android:layout_width="wrap_content"

    android:layout_height="wrap_content"

    android:text="ONNX"

    android:textColor="#000"/>

```

```

<RadioButton

    android:id="@+id/radioStylize"

    android:layout_width="wrap_content"

    android:layout_height="wrap_content"

    android:text="Stylize"

    android:textColor="#000"/>

</RadioGroup>

<!-- Input Source -->

<RadioGroup

    android:id="@+id/inputSourceGroup"

    android:layout_width="0dp"

    android:layout_height="wrap_content"

    android:orientation="horizontal"

    android:gravity="center"

    app:layout_constraintTop_toBottomOf="@id/modeGroup"

    app:layout_constraintStart_toStartOf="parent"

    app:layout_constraintEnd_toEndOf="parent"

    android:layout_marginTop="8dp">

<RadioButton

    android:id="@+id/radioUpload"

    android:layout_width="wrap_content"

```

```

        android:layout_height="wrap_content"

        android:text="Upload"

        android:textColor="#000"/>

<RadioButton

        android:id="@+id/radioCapture"

        android:layout_width="wrap_content"

        android:layout_height="wrap_content"

        android:text="Capture"

        android:textColor="#000"/>

<RadioButton

        android:id="@+id/radioLive"

        android:layout_width="wrap_content"

        android:layout_height="wrap_content"

        android:text="Go Live"

        android:textColor="#000"/>

</RadioGroup>

<!-- Button Panel (Start / Save / Capture) -->

<LinearLayout

        android:id="@+id/buttonPanel"

        android:layout_width="wrap_content"

        android:layout_height="wrap_content"

```

```
android:orientation="vertical"

android:gravity="center"

app:layout_constraintTop_toBottomOf="@id/inputSourceGroup"

app:layout_constraintStart_toStartOf="parent"

app:layout_constraintEnd_toEndOf="parent"

android:layout_marginTop="16dp">
```

```
<Button
```

```
    android:id="@+id/btnChoose"

    android:layout_width="wrap_content"

    android:layout_height="wrap_content"

    android:text="Start 🎬"

    android:textAllCaps="false"

    android:padding="12dp"

    android:backgroundTint="#4A90E2"

    android:textColor="#FFF"/>
```

```
<Button
```

```
    android:id="@+id/btnSave"

    android:layout_width="wrap_content"

    android:layout_height="wrap_content"

    android:text="Save 📁"

    android:textAllCaps="false"
```

```

        android:backgroundTint="#4CAF50"

        android:textColor="#FFF"

        android:layout_marginTop="8dp"/>

<!-- Capture button only visible when preview is bound for capture -->

<Button

    android:id="@+id/btnCapture"

    android:layout_width="wrap_content"

    android:layout_height="wrap_content"

    android:text="Capture"

    android:textAllCaps="false"

    android:backgroundTint="#FF9800"

    android:textColor="#FFF"

    android:visibility="gone"

    android:layout_marginTop="8dp"/>

</LinearLayout>

<!-- Stop button (separate so we can show/hide it while hiding the buttonPanel) -->

<Button

    android:id="@+id/btnStop"

    android:layout_width="wrap_content"

    android:layout_height="wrap_content"

    android:text="Stop Live"

```

```

        android:textAllCaps="false"

        android:backgroundTint="@android:color/holo_red_dark"

        android:textColor="@android:color/white"

        android:visibility="gone"

        app:layout_constraintTop_toBottomOf="@id/buttonPanel"

        app:layout_constraintStart_toStartOf="parent"

        app:layout_constraintEnd_toEndOf="parent"

        android:layout_marginTop="8dp"/>
</androidx.constraintlayout.widget.ConstraintLayout>

```

Item_page.xml:

```

<?xml version="1.0" encoding="utf-8"?>

<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"

        android:layout_width="match_parent"

        android:layout_height="match_parent">

    <ImageView

        android:id="@+id/imageView"

        android:layout_width="match_parent"

        android:layout_height="match_parent"

        android:scaleType="fitCenter" />

</FrameLayout>

```


AndriodManifest.xml:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"

    package="com.example.ribhance">

    <uses-permission android:name="android.permission.CAMERA"/>

    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>

    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"

        android:maxSdkVersion="28"/>

    <application

        android:allowBackup="true"

        android:label="Ribhnace"

        android:icon="@mipmap/ic_launcher"

        android:theme="@style/Theme.AppCompat.Light.NoActionBar">

        <activity android:name=".MainActivity" android:exported="true">

            <intent-filter>

                <action android:name="android.intent.action.MAIN"/>

                <category android:name="android.intent.category.LAUNCHER"/>

            </intent-filter>

        </activity>

    </application>

</manifest>
```

Build.gradle.kts(:app):

```
plugins {  
    id("com.android.application")  
}  
  
android {  
    namespace = "com.example.ribhance"  
    compileSdk = 34  
  
    defaultConfig {  
        applicationId = "com.example.ribhance"  
        minSdk = 24  
        targetSdk = 34  
        versionCode = 1  
        versionName = "1.0"  
  
        ndk {  
            abiFilters += listOf("armeabi-v7a", "arm64-v8a")  
        }  
    }  
  
    buildTypes {  
        release {  
            isMinifyEnabled = false  
            proguardFiles(  
                getDefaultProguardFile("proguard-android-optimize.txt"),  
                "proguard-rules.pro"  
            )  
        }  
    }  
  
    compileOptions {  
        sourceCompatibility = JavaVersion.VERSION_11  
        targetCompatibility = JavaVersion.VERSION_11  
    }  
}  
  
dependencies {  
    // AndroidX UI  
    implementation("androidx.appcompat:appcompat:1.7.0")  
    implementation("com.google.android.material:material:1.12.0")  
}
```

```
implementation("androidx.constraintlayout:constraintlayout:2.1.4")
implementation("androidx.viewpager2:viewpager2:1.1.0")
```

```
// CameraX
```

```
val cameraxVersion = "1.3.3"
```

```
implementation("androidx.camera:camera-core:$cameraxVersion")
```

```
implementation("androidx.camera:camera-camera2:$cameraxVersion")
```

```
implementation("androidx.camera:camera-lifecycle:$cameraxVersion")
```

```
implementation("androidx.camera:camera-view:$cameraxVersion")
```

```
implementation("androidx.camera:camera-extensions:$cameraxVersion")
```

```
// OpenCV (module you imported)
```

```
implementation(project(":opencv"))
```

```
// ONNX Runtime (AI models)
```

```
implementation("com.microsoft.onnxruntime:onnxruntime-android:1.19.0")
```

```
}
```

build.gradle.kts(project:Ribhance):

```
// Root build.gradle.kts
```

```
plugins {
```

```
    id("com.android.application") version "8.11.2" apply false
```

```
    id("com.android.library") version "8.11.2" apply false
```

```
}
```

Build.gradle(:opencv):

```
// This file is part of OpenCV project.
// It is subject to the license terms in the LICENSE file found in the top-level directory
// of this distribution and at http://opencv.org/license.html.

//
// Notes about integration OpenCV into existed Android Studio application project are below
(application 'app' module should exist).
//
// This file is located in <OpenCV-android-sdk>/sdk directory (near 'etc', 'java', 'native'
subdirectories)
//
// Add module into Android Studio application project:
//
// - Android Studio way:
// (will copy almost all OpenCV Android SDK into your project, ~200Mb)
//
// Import module: Menu -> "File" -> "New" -> "Module" -> "Import Gradle project":
// Source directory: select this "sdk" directory
// Module name: ":opencv"
//
// - or attach library module from OpenCV Android SDK
// (without copying into application project directory, allow to share the same module between
projects)
//
// Edit "settings.gradle" and add these lines:
//
// def opencvSdk='<path_to_opencv_android_sdk_rootdir>'
// // You can put declaration above into gradle.properties file instead (including file in HOME
directory),
// // but without 'def' and apostrophe symbols ('):
opencvSdk=<path_to_opencv_android_sdk_rootdir>
// include ':opencv'
// project(':opencv').projectDir = new File(opencvSdk + '/sdk')
//
//
//
// Add dependency into application module:
//
// - Android Studio way:
// "Open Module Settings" (F4) -> "Dependencies" tab
```

```

//
// - or add "project(':opencv')" dependency into app/build.gradle:
//
// dependencies {
//     implementation fileTree(dir: 'libs', include: ['*.jar'])
//     ...
//     implementation project(':opencv')
// }
//
//
// Load OpenCV native library before using:
//
// - avoid using of "OpenCVLoader.initAsync()" approach - it is deprecated
// It may load library with different version (from OpenCV Android Manager, which is installed
// separately on device)
//
// - use "System.loadLibrary("opencv_java4")" or "OpenCVLoader.initDebug()"
// TODO: Add accurate API to load OpenCV native library
//
//
// Native C++ support (necessary to use OpenCV in native code of application only):
//
// - Use find_package() in app/CMakeLists.txt:
//
// find_package(OpenCV 4.12 REQUIRED java)
// ...
// target_link_libraries(native-lib ${OpenCV_LIBRARIES})
//
// - Add "OpenCV_DIR" and enable C++ exceptions/RTTI support via app/build.gradle
// Documentation about CMake options: https://developer.android.com/ndk/guides/cmake.html
//
// defaultConfig {
//     ...
//     externalNativeBuild {
//         cmake {
//             cppFlags "-std=c++11 -frtti -fexceptions"
//             arguments "-DOpenCV_DIR=" + opencvSdk + "/sdk/native/jni" //, "-
DANDROID_ARM_NEON=TRUE"
//         }
//     }
// }

```

```
// }
//
// - (optional) Limit/filter ABIs to build ('android' scope of 'app/build.gradle'):
// Useful information: https://developer.android.com/studio/build/gradle-tips.html (Configure
// separate APKs per ABI)
//
// splits {
//     abi {
//         enable true
//         universalApk false
//         reset()
//         include 'armeabi-v7a' //, 'x86', 'x86_64', 'arm64-v8a'
//     }
// }
//
```

```
apply plugin: 'com.android.library'
apply plugin: 'maven-publish'
try {
    apply plugin: 'kotlin-android'
    println "Configure OpenCV with Kotlin"
} catch (Exception e) {
    println "Configure OpenCV without Kotlin"
}
```

```
def openCVersionName = "4.12.0"
def openCVersionCode = ((4 * 100 + 12) * 100 + 0) * 10 + 0
```

```
println "OpenCV: " + openCVersionName + " " + project.buildscript.sourceFile
```

```
android {
    namespace 'org.opencv'
    compileSdkVersion 34
```

```
    defaultConfig {
        minSdkVersion 21
        targetSdkVersion 34
```

```
    externalNativeBuild {
        cmake {
            arguments "-DANDROID_STL=c++_shared"
```

```

        targets "opencv_jni_shared"
    }
}

android {
    buildFeatures {
        buildConfig true
    }
}

compileOptions {
    sourceCompatibility JavaVersion.VERSION_17
    targetCompatibility JavaVersion.VERSION_17
}

buildTypes {
    debug {
        packagingOptions {
            doNotStrip '**/*.so' // controlled by OpenCV CMake scripts
        }
    }
    release {
        packagingOptions {
            doNotStrip '**/*.so' // controlled by OpenCV CMake scripts
        }
        minifyEnabled false
        proguardFiles getDefaultProguardFile('proguard-android.txt'), 'proguard-rules.txt'
    }
}

sourceSets {
    main {
        jniLibs.srcDirs = ['native/libs']
        java.srcDirs = ['java/src']
        res.srcDirs = ['java/res']
        manifest.srcFile 'java/AndroidManifest.xml'
    }
}

externalNativeBuild {
    cmake {
        path (project.projectDir.toString() + '/libcxx_helper/CMakeLists.txt')
    }
}

```

```

    }

    buildFeatures {
        prefabPublishing true
        buildConfig true
    }
    prefab {
        opencv_jni_shared {
            headers 'native/jni/include'
        }
    }

    publishing {
        singleVariant('release') {
            withSourcesJar()
            withJavadocJar()
        }
    }
}

publishing {
    publications {
        release(MavenPublication) {
            groupId = 'org.opencv'
            artifactId = 'opencv'
            version = '4.12.0'
            afterEvaluate {
                from components.release
            }
        }
    }
    repositories {
        maven {
            name = 'myrepo'
            url = "${project.buildDir}/repo"
        }
    }
}

dependencies {
}

```


Settings.gradle.kts(Ribhance):

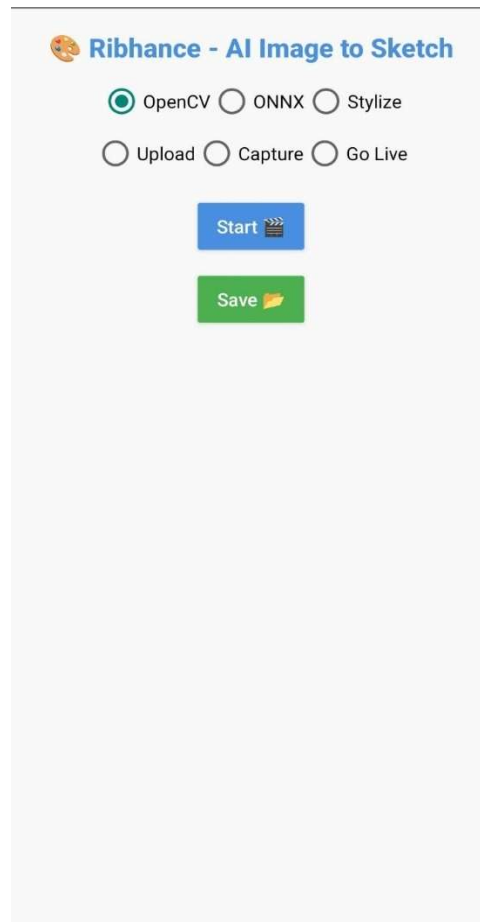
```
pluginManagement {  
    repositories {  
        gradlePluginPortal()  
        google()  
        mavenCentral()  
    }  
}  
  
dependencyResolutionManagement {  
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)  
    repositories {  
        google()  
        mavenCentral()  
    }  
}  
  
rootProject.name = "Ribhance"  
include(":app")  
// If you import OpenCV as a module, uncomment the next line after importing:  
// include(":opencv")  
include(":opencv")  
project(":opencv").projectDir = File(rootDir, "opencv-4.12.0-android-sdk/OpenCV-android-sdk/sdk/")
```

CHAPTER-7

SCREENSHOTS

7.SCREENSHOTS

This section presents various screenshots of the “Ribhance – AI Image to Sketch” application. Each screenshot represents different modules and functionalities implemented in the system, demonstrating the user interface and workflow.



7.1 Home Page

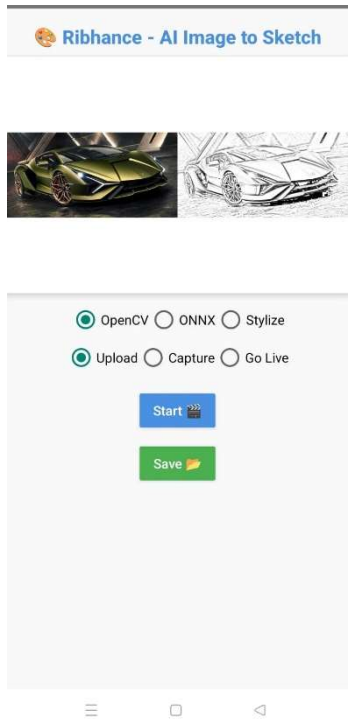
The Home Page is the first screen displayed to the user after launching the application. It provides an intuitive and simple interface with options to choose the **processing method** (OpenCV or ONNX) and **input mode** (Upload, Capture, or Go Live). Users can easily navigate to different functionalities from this screen.

Module Selection & Navigation

The application provides smooth navigation between modes. From the Home Page, users can switch between Upload, Capture, and Live modes at any time. Each mode maintains a clear layout and responsive buttons, ensuring a user-friendly experience.

7.2 Image upload mode

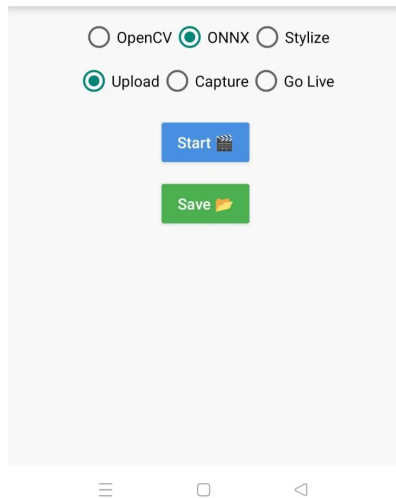
In this module, users can upload any existing image from their device's gallery. Once the image is selected, the application processes it using the chosen AI model (ONNX or OpenCV), stylize mode (optional) and displays the sketch output on the screen. This mode is useful for processing already captured images.



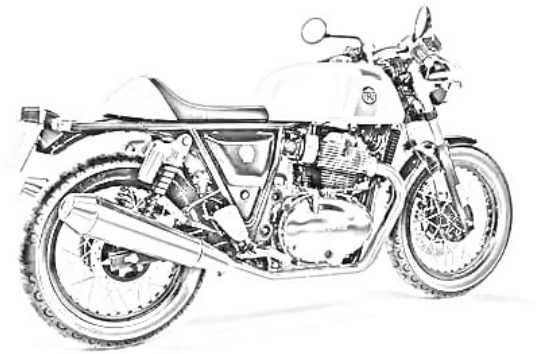
7.2.1 OpenCV Mode



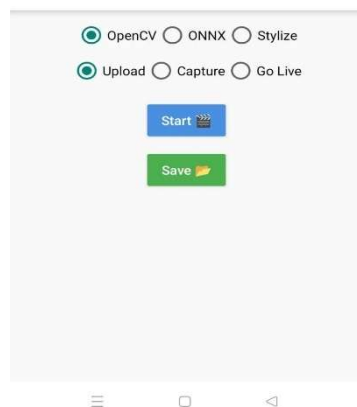
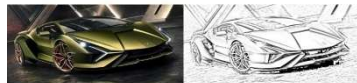
7.2.1.1 Saved Image



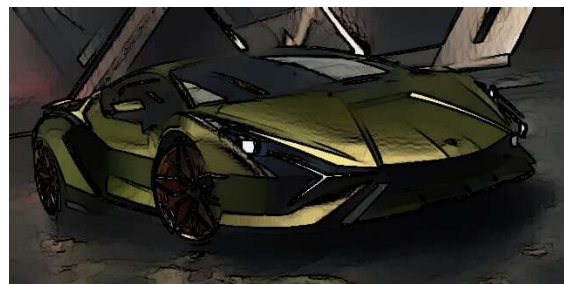
7.2.2 ONNX Mode



7.2.2.1 Saved Image



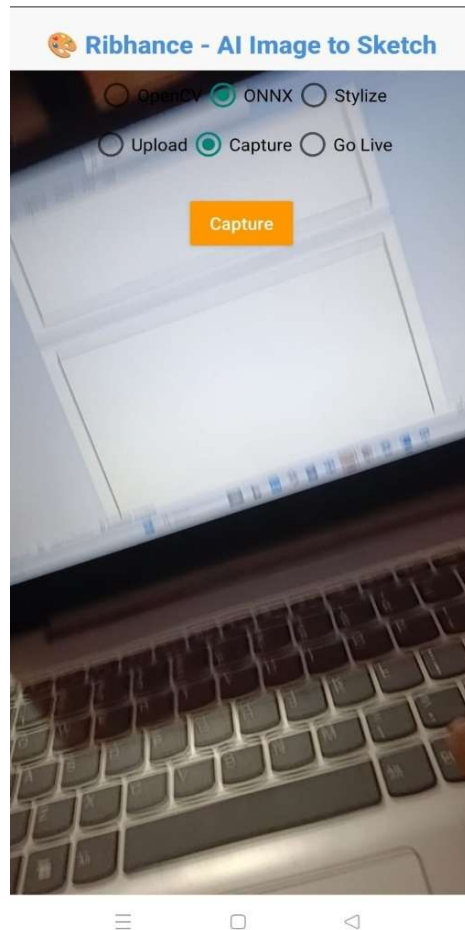
7.2.3 Stylize Mode(optional)



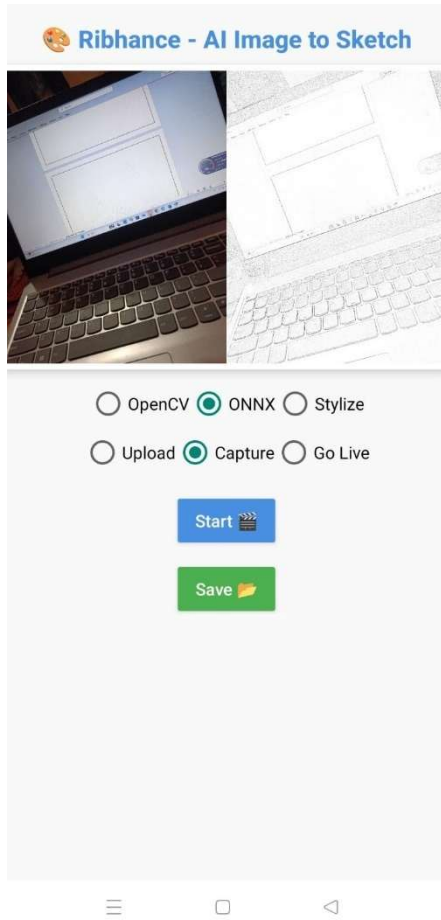
7.2.3.1 Saved Image

7.3 One-Time Capture Mode

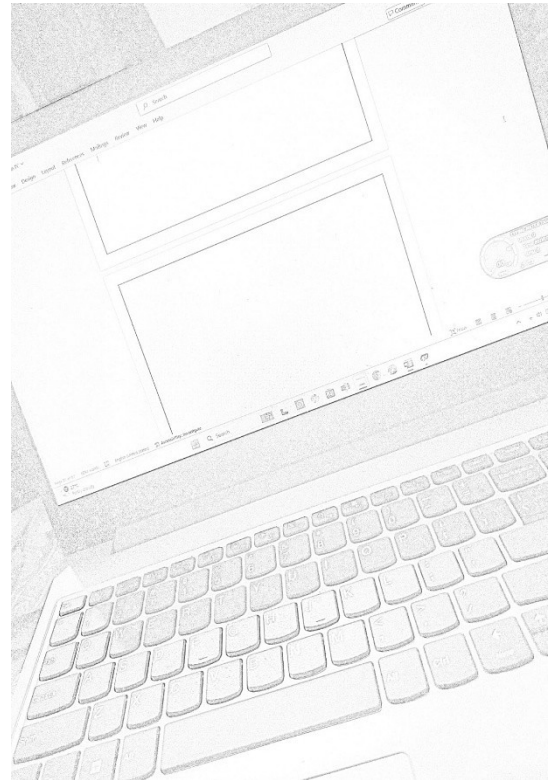
The Capture Mode allows users to take a photo directly using the device's camera. After capturing, the image is instantly processed and converts the image into a high-quality pencil sketch. This mode provides a quick way to sketch real-time photos without needing to upload them separately.



7.3.1 Capture Screen



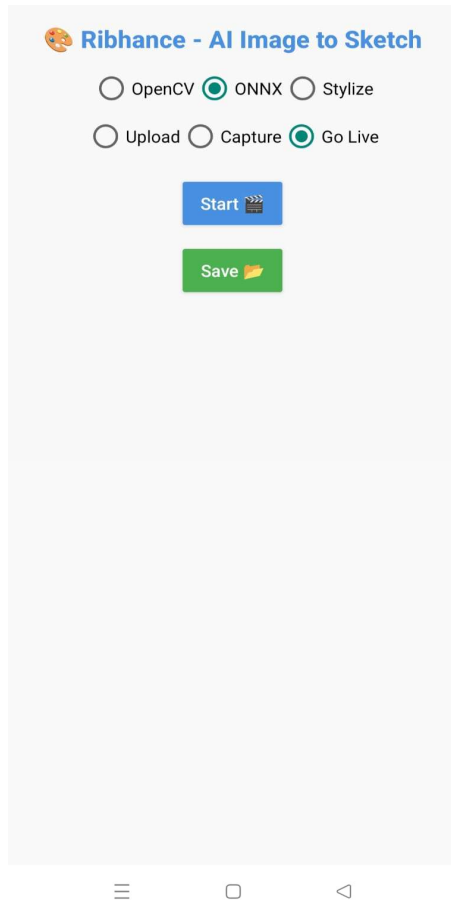
7.3.2 Capture Mode



7.3.3 Saved Image

7.4 Live Mode (ONNX)

In Live Mode, the camera feed is continuously processed using the ONNX model. Users can see real-time sketch transformations of whatever the camera captures. This mode is particularly useful for live demonstrations and interactive sketching effects.



7.4.1 Live Mode



7.4.2 Live Preview

CHAPTER-8

CONCLUSION

8.CONCLUSION

The *AI Image to Sketch Application* has been successfully developed using Java, Kotlin (DSL), and XML in Android Studio, integrating OpenCV and ONNX (TensorFlow Lite/AI models) for real-time and static image sketch generation. The system allows users to upload images, capture photos, or use live camera streaming to generate artistic sketch outputs efficiently.

This project demonstrates the effective combination of mobile computing, computer vision, and artificial intelligence to achieve lightweight, fast, and high-quality image-to-sketch transformation directly on Android devices without requiring cloud processing. The modular design ensures scalability, enabling future integration of advanced AI models or new artistic filters.

From a technical perspective, the implementation of preprocessing, model inference, and postprocessing pipelines has shown how image data can be efficiently handled and transformed on resource-constrained environments like smartphones. Moreover, storing processed images in the gallery makes the system practical and user-friendly for real-world usage.

In conclusion, the project proves that AI-based image stylization can be effectively deployed in mobile platforms, offering both innovation and usability. It not only enhances user experience with creative applications but also establishes a foundation for further research in areas like mobile AI optimization, artistic rendering, and real-time computer vision systems.

Chapter-9

FUTURE SCOPE

9.FUTURE SCOPE

The *AI Image to Sketch Application* can be further enhanced and expanded in several ways:

1. Integration of Advanced AI Models

- Deploy lightweight GAN-based models or transformer-based architectures for more realistic and diverse sketch styles.
- Enable style transfer options where users can choose between multiple artistic effects.

2. Cross-Platform Support

- Extend the application to iOS and web platforms for broader accessibility.
- Provide synchronization with cloud services so users can store and retrieve their creations seamlessly.

3. Performance Optimization

- Implement quantization and pruning techniques to reduce model size and inference time.
- Utilize GPU acceleration (via Vulkan or Metal APIs) for smoother live rendering.

4. Enhanced User Features

- Add editing tools (brushes, filters, contrast adjustments) so users can refine generated sketches.
- Provide social media integration for one-click sharing.
- Include batch processing, allowing users to sketchify multiple images at once.

5. Research and Innovation

- Explore real-time video stylization, turning live camera streams into sketch-style videos.
- Integrate augmented reality (AR) features, where sketches can be overlaid in real-world environments.

CHAPTER-10

REFERENCES

10.REFERENCES

1. <https://developer.android.com/studio> --Andriod Studio Installation
2. <https://onnxruntime.ai> – For ONNX Runtime Module
3. <https://opencv.org/releases/> --For OpenCV Module
4. <https://github.com/bhanusaiyarasu/imgtosketchproject>
--Visit the above link for App, Code and other resources